

Chaos Engineering for AI Systems

Breaking things on purpose to build resilience

A systematic approach to testing AI system resilience by deliberately introducing failures to build more robust production environments.



Why Chaos Engineering?



Complex & Distributed

AI infrastructure spans multiple systems, services, and dependencies, creating numerous potential failure points.



Inevitable Failures

Nodes crash, networks partition, models stall—failures are a matter of when, not if.



Proactive Confidence

Simulate failures **before** they happen in production to improve reliability and build team confidence.

Core Principles of Chaos Engineering

Define Steady State

Establish normal operational metrics for your AI system:

- Inference latency (p95, p99)
- Throughput (requests per second)
- Model accuracy/performance
- Resource utilization baselines

Observe System Response

Monitor how your system handles the failure and measure deviation from steady state.

Inject Controlled Failure

Deliberately introduce failures with clear scope and abort conditions.

Improve & Repeat

Fix vulnerabilities, strengthen resilience, and run progressively more complex experiments.

Each cycle builds greater confidence in your AI system's reliability under stress.

Chaos Targets in AI Systems



Infrastructure

GPU node crash, pod eviction, resource exhaustion, zone failure

Network

Latency injection, packet loss, DNS failures, API timeouts

Data Pipeline

Delayed feature ingestion, schema changes, corrupted batches

Model Serving

OOM errors, unresponsive inference pods, model version conflicts

Tools for Chaos Engineering

1

Chaos Mesh

Kubernetes-native chaos engineering platform that orchestrates chaos experiments in cloud-native environments.

- Pod/node failures
- Network chaos
- I/O chaos

2

LitmusChaos

Cloud-native chaos toolkit that provides pre-built chaos experiments for Kubernetes and cloud infrastructure.

- Application-level faults
- Node-level faults

3

Gremlin

Commercial chaos-as-a-service platform with a user-friendly interface and sophisticated experiment controls.

- Fine-grained blast radius
- Advanced scheduling

4

Istio/Envoy

Service mesh tools that enable API-level chaos through fault injection at the request level.

- HTTP error codes
- Request delays

Example: Pod Kill Experiment

Experiment Setup

1. Establish baseline: 50 QPS inference with $p95 < 200\text{ms}$
2. Define hypothesis: "Our system can handle random pod terminations without service degradation"
3. Inject chaos: Use Chaos Mesh to randomly kill inference pods
4. Monitor: Track latency, throughput, error rates in real-time
5. Validate: Kubernetes reschedules pods → traffic reroutes → steady state maintained

❏ **Success criteria:** QPS remains steady, p95 latency stays below threshold despite pod failures



Real-time metrics during pod failure experiment



Example: GPU Failure Simulation



Remove GPU Device Plugin

Simulate hardware failure by removing the device plugin from a node, making GPUs unavailable



Observe Rescheduling

Kubernetes should detect the issue and reschedule workloads to healthy GPU nodes



Verify Checkpointing

Training jobs should checkpoint and resume without data loss



Confirm Resilience

Measure recovery time and validate training progress continuity

This experiment **proves your training jobs are fault-tolerant** and can survive GPU hardware failures with minimal disruption.

Chaos for Model Pipelines

Pipeline Resilience Tests

Data Ingestion Delays

Inject delayed Kafka messages to test feature pipeline robustness

Corrupted Data Injection

Introduce bad data batches to verify drift detection mechanisms

Partial Processing Failures

Kill ETL jobs mid-processing to test recovery capabilities

Expected Outcomes

- **Auto-recovery** of pipeline components
- **Retraining triggers** activate correctly when thresholds are crossed
- **Data validation** catches and quarantines problematic inputs
- **Alerting systems** notify appropriate teams
- **Self-healing workflows** maintain model quality

Building confidence in your ML system's ability to maintain data quality under stress.

Best Practices

Start Small, Scale Gradually

Begin in staging environments with simple experiments, then cautiously expand to production with increasing complexity.

Example: Start with single pod failures before testing multi-node outages

Automate Your Chaos

Integrate chaos experiments into CI/CD pipelines for continuous reliability testing.

Example: Run pod failure tests after each deployment

Define Clear Abort Conditions

Always have predetermined thresholds that automatically stop experiments if critical services are impacted.

Example: Abort if error rate exceeds 0.5% or p99 latency > 500ms

Measure MTTR

Track Mean Time To Recovery as your key reliability metric and work to improve it over time.

Example: Target 30-second recovery for inference service disruptions

"Chaos engineering is about building confidence in your system's resilience through experimentation."

AI-Specific Challenges

Beyond Uptime Metrics

Must measure model accuracy and fairness under stress, not just system availability

Silent Drift Detection

Failures may cause gradual performance degradation rather than immediate crashes

Complex Blast Radius

Multi-tenant GPU clusters require careful experiment isolation to prevent collateral damage

Integrated Observability

Need to combine infrastructure metrics with ML-specific telemetry for complete visibility



Combined infrastructure and ML observability dashboard

❌ **Unique AI Risk:** A system can appear healthy from an infrastructure perspective while delivering increasingly inaccurate predictions.

Key Takeaways

1 Proactive Reliability Testing

Chaos engineering identifies weaknesses before they impact users by deliberately introducing controlled failures.

2 AI-Specific Chaos Targets

Focus on pods, GPUs, pipelines, and model serving components unique to AI infrastructure.

3 Tooling Ecosystem

Leverage Chaos Mesh, LitmusChaos, Gremlin, and service mesh capabilities for comprehensive testing.

4 Confidence Builder

Regular chaos testing builds engineer confidence in the system's resilience to real-world failures.

Break things intentionally

So they don't break unexpectedly

Start your chaos journey today!

